

南开大学

信息安全综合实验课程实验报告

RegScope 项目



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

一、选题背景

在恶意代码分析与防治课程中，我们曾使用 Regshot 这一工具进行注册表分析，Regshot 通过拍摄两次注册表快照并对比，最后给出修改的注册表键和值。这一功能对于恶意代码分析具有重要意义，但是 Regshot 仅支持 32 位 Windows 系统，并且其 UI 界面较为简单，最重要的是它只能实现两个时间点的注册表监控，不能实现一段时间内持续的注册表监控。因此，我们决定在其源码基础上进行修改，改进前端界面并增加持续监控功能，为用户展现恶意代码行为中更多的细节。

二、环境配置

1. Qt Creator

在这次大作业中我们使用 Qt Creator 作为开发工具。Qt Creator 是一个专为 Qt 框架设计的跨平台集成开发环境，支持在 Windows、Linux、macOS 上运行，并能将代码部署到桌面、Android、iOS 及嵌入式平台。它整合了可视化界面设计器、内置帮助文档和翻译工具提供智能代码编辑、CMake/qmake 项目构建管理以及 Git 等版本控制系统集成，能帮助开发者高效完成跨平台图形界面应用程序的开发。

2. Regshot

Regshot 是一款小巧、免费且开源的注册表比较工具，主要用于捕获和比较系统注册表及指定文件夹的快照。其核心用途是在进行系统更改或安装卸载软件前后，分别创建两份快照并自动进行对比，最终生成详细的文本或 HTML 报告，列出所有新增、删除或修改的注册表项及文件变化。

我们从 github 上获取了 Regshot 的源码进行分析和研究，并在此基础上进行修改。修改的方面包括实现基本功能、优化逻辑、适配 64 位 Windows 系统等。

3. 小组分工

我们小组的成员是段俊宇和陈展，我主要负责快照拍摄、比较等功能的实现以及注册表过滤项的设置，陈展同学负责整体框架、前端逻辑以及 ETW 持续化监控的实现。

三、开发环境

Windows 11 操作系统，Qt Creator 6.11.0

四、本人分工描述

由于我主要负责的是后端代码的实现，因此接下来我将结合不同函数的代码进行逻辑和功能解释。

1. 结构体设计

参考 Regshot 的代码，我设计了下列结构体，我将使用注释的方式进行解释。

```
1. // 计数结构体
2. struct _COUNTS
3. {
4.     DWORD cAll;
5.     DWORD cKeys;
6.     DWORD cValues;
7. };
8. typedef _COUNTS Counts, *LPCounts;
```

该结构体用于统计注册表键和注册表值修改的数量，这样可以在比较快照时清晰展现所有的更改。

```
1. // 注册表值结构体
2. struct _VALUE
3. {
4.     LPTSTR lpValueName;           // 指向注册表值名称
5.     DWORD dwValueNameLen;         // 注册表值名称缓冲区大小
6.     DWORD dwTypeCode;             // 注册表值类型，REG_DWORD、REG_SZ..
7.     LPBYTE lpValueData;          // 注册表值数据
8.     DWORD dwDataSize;             // 注册表值数据大小
9.     struct _KEY *lpFatherKey;      // 指向注册表值的父键
10.    struct _VALUE *lpBrotherValue; // 指向兄弟结点注册表值
11.    DWORD fValueMatch;             // 该注册表值是否匹配
12. };
13. typedef struct _VALUE ValueContent, *LPValueContent;
```

该结构体是注册表值结构体，包含了值名称、数据相关的成员。指向

注册表值名称的指针和注册表值名称缓冲区搭配一起使用可以正确读取注册表值的名称，值包含的数据也是如此。指向注册表值的父键能够在遍历注册表的时候作为循环入口，而指向兄弟结点注册表值可以作为下一循环的入口以及跳出循环的出口，当该指针为空时代表是最后一个值。注册表值是否匹配这一变量用于比较快照时使用，具体有修改、匹配和未匹配三种状态。

```
1. // 注册表键结构体
2. struct _KEY
3. {
4.     LPTSTR lpKeyName;           // 指向注册表键名称
5.     DWORD dwKeyNameLen;         // 注册表键名称缓冲区大小
6.     LPValueContent lpFirstValue; // 指向该注册表键的第一个值
7.     struct _KEY *lpFirstSubKey;  // 指向第一个子键
8.     struct _KEY *lpBrotherKey;   // 指向兄弟结点注册表键
9.     struct _KEY *lpFatherKey;    // 指向父结点注册表键
10.    DWORD fKeyMatch;             // 该注册表键是否匹配
11. };
12. typedef struct _KEY KeyContent, *LPKeyContent;
```

该结构体是注册表值结构体，成员构成与注册表值相似，同样采用树状结构，这里就不再赘述。

```
1. // 注册表快照结构体
2. struct _REGSHOT
3. {
4.     LPKeyContent lpHKCR;        // 指向 HKEY_CLASSES_ROOT
5.     LPKeyContent lpHKCU;        // 指向 HKEY_CURRENT_USER
6.     LPKeyContent lpHKLM;        // 指向 HKEY_LOCAL_MACHINE
7.     LPKeyContent lpHKU;         // 指向 HKEY_USERS
8.     LPKeyContent lpHKCC;        // 指向 HKEY_CURRENT_CONFIG
9.     bool fFinished;             // 是否拍摄快照完成
10.    bool fLoaded;                // 是否加载完成
11.    LPTSTR lpShotName;           // 指向快照名称
12.    LPTSTR lpComputerName;       // 计算机名称
13.    LPTSTR lpUserName;           // 用户名称
14.    quint32 dwComputerName;      // 计算机名称长度
15.    quint32 dwUserName;          // 用户名称长度
16.    DWORD i;                    // 快照编号
17.    Counts SumCounts;            // 遍历的总数
18.    SYSTEMTIME SystemTime;       // 快照保存时间
19. };
20. typedef struct _REGSHOT RegShotContent, *LPRegShotContent;
```

该结构体是注册表快照结构体，包含一系列成员变量。由于我们的注册表快照包含五种注册表根键，因此成员中设置了指向他们的指针。fFinished 和 fLoaded 均用于向前端传递信息，拍摄完成和加载完成时会在前端输出相应的信息。其他的成员变量用于设置快照的基本信息，例如计算机名称、快照保存时间等。

```
1. // 保存值结构体
2. struct _SAVEVALUE
3. {
4.     quint64 ofsValueName;    // 值名称的偏移量
5.     quint64 ofsValueData;    // 值数据的偏移量
6.     quint64 ofsBrotherValue; // 兄弟结点注册表值的偏移量
7.     quint64 ofsFatherKey;    // 注册表值的父键的偏移量
8.     quint32 dwTypeCode;      // 值类型
9.     quint32 dwDataSize;      // 值数据的大小
10.    quint32 dwValueNameLen;   // 值名称的长度
11. };
12. typedef struct _SAVEVALUE SaveValueContent, *LPSaveValueContent;
```

该结构体是保存注册表值的结构体，其中包含和偏移量相关的成员。由于快照文件采用二进制写入，需要通过偏移量标识不同数据的位置，因此设置了注册表值名称、值数据、兄弟结点及父键的偏移量。

```
1. // 保存键结构体
2. struct _SAVEKEY
3. {
4.     quint64 ofsKeyName;      // 键名称的偏移量
5.     quint64 ofsFirstValue;   // 第一个值的偏移量
6.     quint64 ofsFirstSubKey;   // 第一个子键的偏移量
7.     quint64 ofsBrotherKey;    // 兄弟结点注册表键的偏移量
8.     quint64 ofsFatherKey;    // 父结点注册表键的偏移量
9.     quint32 dwKeyNameLen;     // 键名称的长度
10. };
11. typedef struct _SAVEKEY SaveKeyContent, *LPSaveKeyContent;
```

该结构体是保存注册表键的结构体，成员构成与保存注册表值的结构体相似，这里就不再赘述。

```
1. // 文件头部信息结构体
2. struct _FILEHEADER
3. {
4.     char signature[8];       // 魔数
5.     quint64 ofsHKCR;         // HKCR 数据位置
```

```

6.    quint64 ofsHKCU;           // HKCU 数据位置
7.    quint64 ofsHKLM;           // HKLM 数据位置
8.    quint64 ofsHKU;            // HKU 数据位置
9.    quint64 ofsHKCC;           // HKCC 数据位置
10.   quint32 dwEndianness;       // 字节序
11.   SYSTEMTIME SystemTime;      // 快照保存时间
12.   quint64 ofsComputerName;    // 计算机名称偏移量
13.   quint32 dwComputerNameLen;  // 计算机名称长度
14.   quint64 ofsUserName;        // 用户名称偏移量
15.   quint32 dwUserNameLen;      // 用户名称长度
16. };
17. typedef _FILEHEADER FileHeader, *LPFileheader;

```

该结构体是快照头部结构体，同样采用偏移量的方式设置成员。此外，这里还设置了文件的魔数“REGSCOPE”用于标识快照文件，将其与其他类型的文件进行区分。

```

1. // 注册表排除项结构体
2. struct _REGSKIPITEM
3. {
4.     LPTSTR FullPath; // 排除项的完整路径
5.     bool Active;      // 是否启用排除项，默认为 1
6. };
7. typedef _REGSKIPITEM RegSkipItem;

```

该结构体是注册表排除项结构体，其中包含了排除项的完整路径和启用标志。注册表包含数量庞大的注册表键和值，受到权限和用途的影响，其中很多都与恶意代码的行为无关，对其进行监控会大大拖慢快照拍摄时间，遗漏关键的恶意代码行为，据此设置了排除项。

```

1. // 比较结果结构体
2. struct _COMPARES
3. {
4.     LPVOID lpOld;           // 旧的内容
5.     LPVOID lpNew;           // 新的内容
6.     struct _COMPARES *lpNext; // 指向下一个比较结果
7. };
8. typedef _COMPARES Compares, *LPCompares;

```

该结构体是比较结果结构体，其中包含三个指针，即两次快照中旧的快照内容，新的内容以及指向下一个比较结果的指针。这种结构参考了 Regshot 的设计，它可以让后续的结果插入变得简洁高效。

```

1. // 比较结果指针结构体

```

```

2. struct _COMPARERESULTSPOINTERS
3. {
4.     LPCompares lpKeyDeleted;    // 删除的注册表键
5.     LPCompares lpKeyAdded;      // 新增的注册表键
6.     LPCompares lpValueDeleted;  // 删除的注册表值
7.     LPCompares lpValueAdded;    // 新增的注册表值
8.     LPCompares lpValueModified; // 修改的注册表值
9. };
10. typedef _COMPARERESULTSPOINTERS CompareResultsPointers, *LPCompareResultsPointers;

```

该结构体是比较结果指针结构体，包含了各类变化的指针，能够在比较结果时展现注册表键和值的变化。

```

1. // 统计比较结果结构体
2. struct _COMPARERESULTS
3. {
4.     LPRegShotContent lpShot1;      // 基准快照
5.     LPRegShotContent lpShot2;      // 比较快照
6.     Counts dwCompared;             // 比较的注册表项数量
7.     Counts dwDeleted;              // 删除的注册表项数量
8.     Counts dwAdded;                // 新增的注册表项数量
9.     Counts dwModified;             // 修改的注册表项数量
10.    CompareResultsPointers CRHeads; // 链表头指针集合
11.    CompareResultsPointers CRCurrent; // 当前指针集合
12.    bool fFinished;                 // 是否比较完成
13. };
14. typedef _COMPARERESULTS CompareResults, *LPCompareResults;

```

该结构体是统计比较结果结构体，用于汇总所有的比较结果并生成比较结果报告。我们采用单链表的形式将所有的注册表项和注册表值比较结果串联在一起，同时对三种变化的数量进行统计，最后可以输出详细结果。

```

1. #define REGSHOT_BUFFER_BLOCKSIZE 1024
2. #define MAX_SIGNATURE_LENGTH 8
3. #define RegScopeFileSignature "REGSCOPE"
4. #define MAX_INI_SECTION_CHARS 32768
5.
6. #define NOMATCH 0
7. #define ISMATCH 1
8. #define ISDEL 2
9. #define ISADD 3
10. #define ISMODI 4
11.
12. #define KEYDEL 1

```

```
13. #define KEYADD 2
14. #define VALDEL 3
15. #define VALADD 4
16. #define VALMODI 5
```

这里定义了一些宏常量，包括块大小、标识字段、键修改、键新增、键删除、值新增等。

```
1. LPTSTR lpStringBuffer;           // 字符串缓冲区指针，用于接收键或值的名称
2. LPBYTE lpDataBuffer;             // 数据缓冲区指针，用于接收值的数据
3. DWORD dwSourceSize;              // 资源大小
4. DWORD dwStringBufferSize;        // 字符串缓冲区大小
5. DWORD dwDataBufferSize;          // 数据缓冲区大小
6. SaveKeyContent SaveKC;            // 键保存结果
7. SaveValueContent SaveVC;          // 值保存结果
8. LPTSTR lpOutputPath;             // 输出路径
9. LPTSTR lpLastSaveDir;            // 上次保存的目录
10. LPTSTR lpShotName;              // 默认快照名称
11. FileHeader fileheader;           // 文件头部
12. std::unordered_map<QString, bool> RegSkipList; // 注册表排除项链表
```

上面是一些私有成员变量的定义，包括了缓冲区、保存目录、默认快照名称等。

下面将介绍不同功能的函数，并对其逻辑进行简要梳理。

2. 构造函数和析构函数

```
1. RegShot::RegShot()
2. {
3.     lpStringBuffer = NULL;
4.     lpDataBuffer = NULL;
5.     dwSourceSize = 0;
6.     dwStringBufferSize = 0;
7.     dwDataBufferSize = 0;
8.     ZeroMemory(&CompareResult, sizeof(CompareResult));
9. }
10.
11. RegShot::~RegShot()
12. {
13.     for (int i = 0; i < Shots.size(); i++)
14.     {
15.         LPRegShotContent shot = &Shots[i];
16.         FreeShot(shot);
17.     }
18.     FreeCompareResult();
```



```
19. }
```

这部分是构造函数和析构函数。构造函数对部分变量进行初始化，防止在程序运行时出现混乱。析构函数调用 FreeShot 函数释放快照中的注册表键和值，最后释放比较结果，对占用的内存空间实现释放。

3. 解析配置文件

```
1. // 解析配置文件
2. QMap<QString, QString> RegShot::ParseIniSection(const QString &filePath, LPTSTR
sectionName)
3. {
4.     QMap<QString, QString> result;
5.     // 打开配置文件
6.     QFile file(filePath);
7.     if (!file.open(QIODevice::ReadOnly | QIODevice::Text))
8.     {
9.         return result;
10.    }
11.    // 逐行解析
12.    QString targetSection = QString::fromWCharArray(sectionName);
13.    QTextStream stream(&file);
14.    bool inTargetSection = false;
15.    while (!stream.atEnd())
16.    {
17.        QString line = stream.readLine().trimmed();
18.        // 检查是否进入目标节
19.        if (!inTargetSection && line.startsWith("[")
20.        {
21.            inTargetSection = (line.compare("[ " + targetSection + "]",
Qt::CaseInsensitive) == 0);
22.            continue;
23.        }
24.        // 读取完成直接退出
25.        if (inTargetSection && line.startsWith(";"))
26.        {
27.            break;
28.        }
29.        // 解析目标节中的键值对
30.        if (inTargetSection && !line.isEmpty() && !line.startsWith(";"))
31.        {
32.            int pos = line.indexOf("=");
33.            if (pos > 0)
34.            {
```

```

35.         QString key = line.left(pos);
36.         QString value = line.mid(pos + 1);
37.         result[key] = value;
38.     }
39. }
40. }
41.
42. return result;
43. }

```

这部分代码是对配置文件内容的解析，配置文件中每一部分用 [title] 单独一行隔开，因此根据这一特点进入指定的目标节并逐行读取所有的内容，在系统初始化时使用。

4. 填充排除项

```

1. // 填充排除项链表
2. void RegShot::FillSkipList(const QMap<QString, QString> SkipRegKeyData)
3. {
4.     RegSkipList.clear();
5.     // 预分配空间，提升性能
6.     RegSkipList.reserve(SkipRegKeyData.size());
7.
8.     for (auto it = SkipRegKeyData.begin(); it != SkipRegKeyData.end(); it++)
9.     {
10.         bool isActive = (it.value() == "1");
11.         RegSkipList[it.key()] = isActive;
12.     }
13. }

```

这部分代码用于填充排除项，在 Regshot 中使用链表装填排除项，并遍历链表进行比对。但是 64 位系统的注册表比 32 位系统大得多，为了更高效的查询，我们使用哈希表储存，将查询的平均时间复杂度降低为 $O(1)$ ，可以显著提升快照拍摄及比较速度。

5. 系统数据初始化

```

1. // 加载系统默认数据
2. void RegShot::LoadSystemData(const QMap<QString, QString> SystemData)
3. {
4.     for (auto it = SystemData.begin(); it != SystemData.end(); it++)
5.     {
6.         // 默认快照名称
7.         if (it.key().compare("title", Qt::CaseInsensitive) == 0)

```

```

8.     {
9.         lpShotName = new TCHAR[it.value().length() + 1];
10.        _tcscpy(lpShotName, it.value().toStdWString().c_str());
11.    }
12.    // 默认快照保存目录
13.    if (it.key().compare("basedir", Qt::CaseInsensitive) == 0)
14.    {
15.        lpOutputPath = new TCHAR[it.value().length() + 1];
16.        _tcscpy(lpOutputPath, it.value().toStdWString().c_str());
17.    }
18.    }
19. }
20. // 加载配置文件
21. bool RegShot::LoadSettingsFromIni()
22. {
23.     IniPath = resolveSelectedConfigPath();
24.     FilterIniPath = resolveFilterConfigPath(IniPath);
25.     // 加载系统默认数据
26.     LoadSystemData(ParseIniSection(IniPath, lpSystem));
27.     // 加载排除项
28.     FillSkipList(ParseIniSection(FilterIniPath, lpIniSkipRegKey));
29.
30.     return true;
31. }

```

这部分代码用于加载系统默认数据，设置了默认快照名称和默认保存目录，并调用 ParseIniSection 函数完成系统默认数据（语言、快照标题）和排除项的加载。自此整个系统初始化过程完成。

6. 检查是否为排除项

```

1. // 是否在排除项链表中
2. bool RegShot::IsInSkipList(LPCTSTR lpKeyName)
3. {
4.     QString keyPath = QString::fromWCharArray(lpKeyName);
5.
6.     auto it = RegSkipList.find(keyPath);
7.     // 是否启用
8.     if (it != RegSkipList.end())
9.     {
10.        return it->second;
11.    }
12.
13.    return false;

```

```
14. }
```

这部分代码实现了对排除项的检查，指针指向了注册表键的完整路径，通过在哈希表中查询返回相应的结果，比遍历链表查询更快。

7. 获取键和价值完整名称

```
1. // 获取键完整名称
2. LPTSTR RegShot::GetWholeKeyName(LPKeyContent lpCurrentKey)
3. {
4.     LPKeyContent lpKC;
5.     LPTSTR lpTail;
6.     DWORD KeyNameLen;
7.     LPTSTR lpFullKeyName;
8.     LPTSTR lpKeyName;
9.     // 计算完整键名的长度
10.    KeyNameLen = 0;
11.    for (lpKC = lpCurrentKey; lpKC != NULL; lpKC = lpKC->lpFatherKey)
12.    {
13.        if (lpKC->lpKeyName != NULL)
14.        {
15.            KeyNameLen += lpKC->dwKeyNameLen;
16.        }
17.        // 预留反斜杠位置
18.        KeyNameLen++;
19.    }
20.
21.    if (KeyNameLen == 0)
22.    {
23.        KeyNameLen++;
24.    }
25.
26.    lpFullKeyName = new TCHAR[KeyNameLen];
27.    if (lpFullKeyName)
28.    {
29.        lpTail = &lpFullKeyName[KeyNameLen - 1];
30.        lpTail[0] = (TCHAR)'\0';
31.    }
32.    // 填充键名
33.    for (lpKC = lpCurrentKey; lpKC != NULL; lpKC = lpKC->lpFatherKey)
34.    {
35.        if (lpKC->lpKeyName != NULL)
36.        {
37.            KeyNameLen = lpKC->dwKeyNameLen;
38.            lpKeyName = lpKC->lpKeyName;
```

```

39.         // 还没到根键
40.         if (lpTail - KeyNameLen != NULL)
41.         {
42.             _tcsncpy(lpTail -= KeyNameLen, lpKeyName, KeyNameLen);
43.             // 添加反斜杠
44.             if (lpTail > lpFullKeyName)
45.             {
46.                 *--lpTail = (TCHAR) '\\';
47.             }
48.         }
49.     }
50. }
51. return lpFullKeyName;
52. }

```

这部分代码用于获取注册表键完整名称，通过不断递归直到根键的方式先获取完整名称的长度，然后再分配空间并依次填充键名和反斜杠，最后返回完整名称。获取注册表值名称时先调用该函数，然后添加值名称即可，这里就不再展示。

8. 分配缓冲区

```

1. // 调整缓冲区，模板函数
2. template <typename T>
3. DWORD RegShot::AdjustBuffer(T *&lpBuffer, DWORD dwCurrentSize, DWORD dwRequiredSize,
4. DWORD dwBlockSize)
5. {
6.     // 缓冲区为空，则当前缓冲区大小为 0
7.     if (lpBuffer == NULL)
8.     {
9.         dwCurrentSize = 0;
10.    }
11.    // 需要扩容
12.    if (dwRequiredSize > dwCurrentSize)
13.    {
14.        if (lpBuffer != NULL)
15.        {
16.            delete[] lpBuffer;
17.            lpBuffer = NULL;
18.        }
19.        // 计算大小
20.        if (dwBlockSize <= 1)
21.        {
22.            dwCurrentSize = dwRequiredSize;

```

```
22.     }
23.     else
24.     {
25.         dwCurrentSize = dwRequiredSize / dwBlockSize;
26.         dwCurrentSize *= dwBlockSize;
27.         if (dwRequiredSize > dwCurrentSize)
28.         {
29.             dwCurrentSize += dwBlockSize;
30.         }
31.     }
32.     // 分配空间
33.     lpBuffer = new T[dwCurrentSize];
34. }
35.
36. return dwCurrentSize;
37. }
```

这部分代码用于调整缓冲区大小，先对当前缓冲区和希望的缓冲区大小进行判断，然后按照结果进行调整并返回分配的空间大小。

9. 获取注册表快照

由于篇幅限制，这里不再展示完整代码，只对逻辑进行梳理。GetRegistrySnap 函数是核心函数之一，实现了注册表键快照的获取。由于注册表键中包含子键和值，因此该函数先使用一个额外的块对当前键中的值进行遍历，减少递归次数，然后再遍历子键。

遍历值的时候先对路径进行拼接，然后检查是否为排除项。如果不是，则通过 RegEnumValue 函数枚举键中的值，并通过尾插法将其插入快照链表进行保存。

遍历子键时通过 RegEnumKeyEx 函数获取子键信息，并提前分配子键名称缓冲区。接下来通过 RegOpenKeyEx 打开子键、构建路径并递归调用函数，以此获得子键内部的信息。最后将双重指针设置为兄弟键，用于下一次循环。

在实现该函数时，我先参照 Regshot 中的逻辑实现，但是测试时发现速度会很慢。这是因为 Regshot 在每次检查排除项时都会先调用 GetWholeKeyName 获取键完整路径，然后再检查。但这样会让本就递归的函数额外增加递归的负担，降低拍摄快照的效率。由于函数最初的参数

是根键，从根键开始向下递归，因此在向下递归时通过传递参数的方式即可获得当前键完整的路径，无需反复递归，大大提升了效率。

Shot 函数用于对注册表所有根键拍摄快照，是总体控制函数；而 TakeSnapshot 函数是 API 接口，用于前端调用。

10. 写入缓冲区

```
1. // 写入缓冲区函数
2. void RegShot::WriteFileBuffer(QDataStream &out, qint64 ofsFile, const void *lpData,
DWORD dwDataSize)
3. {
4.     if (dwDataSize == 0 || lpData == nullptr)
5.     {
6.         return;
7.     }
8.
9.     if (ofsFile < 0)
10.    {
11.        out.device()->seek(out.device()->size());
12.    }
13.    else
14.    {
15.        out.device()->seek(ofsFile);
16.    }
17.
18.    out.writeRawData(reinterpret_cast<const char *>(lpData), dwDataSize);
19. }
```

这部分代码基于 Qt 自带的 QDataStream 类实现了对缓冲区的写入，比 Regshot 中通过指针对缓冲区边界进行操作更加简洁，也提升了代码的可读性。

11. 保存快照

由于篇幅限制，这里不再展示完整代码，只对逻辑进行梳理。 SaveRegKeys 函数是核心函数之一，实现了注册表键快照的保存。该函数通过遍历当前键及其兄弟键实现保存，主要分为保存当前键、保存键中的值、递归保存子键三部分。

在保存当前键之前先获取当前文件位置作为起始位置，并将偏移量写回上一个键相应的位置。接着将键名称及父键的偏移量设置为 0，用

于之后写回。保存当前键时先设置键名称长度，然后先将键结构体内容写入文件，最后写入键名称。

保存键中的值时，同样先获取当前位置作为起始位置，然后先将值结构体写入，再依次写入值名称、数据并设置下一个值的位置用于回写兄弟值的偏移量。

递归写入子键后，设置下一个键的位置用于回写兄弟键的偏移量。写入时采用先写入结构体、再写入具体变量名称的设计，这样在读取的时候可以先生读取结构体，获得每个变量的位置，然后再读取变量的内容。

SaveShot 函数用于保存快照所有信息，包括用户名称等，是总体控制函数；SaveSnapShot 函数是 API 接口，用于前端调用。

12. 加载快照

由于篇幅限制，这里不再展示完整代码，只对逻辑进行梳理。LoadRegKeys 函数是核心函数之一，实现了注册表键快照的加载。该函数通过读取保存的快照，依次加载当前键、当前键的值以及子键，和保存快照的函数流程相反，这里就不再赘述。

LoadShot 函数用于加载快照的所有内容并验证快照的有效性，无需 API 接口，前端直接调用即可。

13. 创建比较结果

```
1. void RegShot::CreateNewResult(DWORD ActionType, LPVOID lpOld, LPVOID lpNew)
2. {
3.     // 初始化
4.     LPCompares lpCR = new Compares();
5.     if (lpCR == NULL)
6.     {
7.         return;
8.     }
9.     lpCR->lpOld = lpOld;
10.    lpCR->lpNew = lpNew;
11.    lpCR->lpNext = NULL;
12.
13.    // 根据不同的修改类型添加到对应的链表
14.    switch (ActionType)
15.    {
16.        // 添加到删除的键链表，尾插法
```



```

17.     case KEYDEL:
18.         (CompareResult.CRHeads.lpKeyDeleted == NULL) ?
CompareResult.CRHeads.lpKeyDeleted = lpCR : CompareResult.CRCurrent.lpKeyDeleted->lpNext
= lpCR;
19.         CompareResult.CRCurrent.lpKeyDeleted = lpCR;
20.         break;
21.         // 添加到新增的键链表
22.     case KEYADD:
23.         (CompareResult.CRHeads.lpKeyAdded == NULL) ? CompareResult.CRHeads.lpKeyAdded
= lpCR : CompareResult.CRCurrent.lpKeyAdded->lpNext = lpCR;
24.         CompareResult.CRCurrent.lpKeyAdded = lpCR;
25.         break;
26.         // 添加到删除的值链表
27.     case VALDEL:
28.         (CompareResult.CRHeads.lpValueDeleted == NULL) ?
CompareResult.CRHeads.lpValueDeleted = lpCR :
CompareResult.CRCurrent.lpValueDeleted->lpNext = lpCR;
29.         CompareResult.CRCurrent.lpValueDeleted = lpCR;
30.         break;
31.         // 添加到新增的值链表
32.     case VALADD:
33.         (CompareResult.CRHeads.lpValueAdded == NULL) ?
CompareResult.CRHeads.lpValueAdded = lpCR : CompareResult.CRCurrent.lpValueAdded->lpNext
= lpCR;
34.         CompareResult.CRCurrent.lpValueAdded = lpCR;
35.         break;
36.         // 添加到修改的值链表
37.     case VALMODI:
38.         (CompareResult.CRHeads.lpValueModified == NULL) ?
CompareResult.CRHeads.lpValueModified = lpCR :
CompareResult.CRCurrent.lpValueModified->lpNext = lpCR;
39.         CompareResult.CRCurrent.lpValueModified = lpCR;
40.         break;
41.     }
42. }

```

这部分代码用于创建新的比较结果并插入不同类型的链表，插入时选择尾插法，时间复杂度为 $O(1)$ ，与 Regshot 相同。

14. 比较快照

由于篇幅限制，这里不再展示完整代码，只对逻辑进行梳理。CompareRegKeys 函数是核心函数之一，实现了注册表键快照的比较。该函数先构建第二次快照中当前键的哈希表索引，接下来比较第一次和第

二次快照中的键和值，最后遍历哈希索引表中剩余的键和值并将其设置为新增。

比较两次快照的键和值时，先构建当前键的完整路径、检查是否为排除项。之后在哈希表中查找相同路径的键，如果找到了就使用额外的块先比较值，再递归比较子键，最后移除哈希表中已经比较过的键。如果找不到就表明该键已经被删除。当前循环结束后，再次遍历哈希表，这些剩余的键和值即为新增。比较值的过程与比较键的过程相同，这里就不再赘述。

在该函数的实现过程中，与获取快照相同，使用传递参数的方式减少递归次数。此外，Regshot 使用四重循环的方式进行比较，最外层循环遍历第一次快照的键，第二层循环遍历第二次快照的键，当找到相同的键时，第三层循环遍历第一次快照中该键的值，第四层循环遍历第二次快照中该键的值。这样的比较方式显然不适用于 64 位系统庞大的注册表，因此采用了哈希表索引的方式，将时间复杂度从 $O(n^4)$ 降低为 $O(n^2)$ ，显著提升了比较速度。

CompareShots 函数用于比较初始化所有标记并比较五个根键，是总体控制函数。

15. 释放内存

由于篇幅限制，这里不再展示完整代码，只对逻辑进行梳理。FreeAllValueContents 函数用于释放值及其数据；FreeAllKeyContents 函数用于释放键和值，释放键时使用递归的方式进行；FreeShot 函数用于释放快照；FreeAllCompare 函数是释放比较结果的辅助函数；FreeCompareResult 函数用于释放比较结果。这些函数用于清理程序占用的内存空间，能够很好的完成收尾工作。

16. 排除项设置

设置排除项时，我将 Regshot 已有的排除项逐个进行了检查，将其中不适用于 64 位系统的删除，并细化了范围，例如将 HKEY_CURRENT_USER\Software\Microsoft\Windows 细化为具体的几个排除项，防止恶意代码的行为被排除。但是这样拍摄快照的速度依然较

慢，不能满足持续监控的要求，因此我们又在 ETW 中进行了二次过滤，按照危险等级设置了排除项，危险等级越高就代表和恶意代码的行为关系越密切。用户可以自行修改这些排除项，增加了灵活性。

以上就是本次信息安全综合实验大作业中我所有工作的汇总！

五、实验结论及心得体会

本次大作业我和陈展同学共同重写了 Regshot 的代码，在原有功能基础上增加了新的功能。在编写代码的过程中，我们遇见了很多的问题，包括运行速度慢、过滤项数量不足、环境配置问题等，但最后我们都将其一一解决。通过本次大作业，我对 Regshot 的功能和代码架构有了非常细致的了解，同时也了解了很多和注册表相关的知识，提升了我的代码能力，扩展了网络安全相关的知识！